

WATERMARK FORGERY ATTACK

Trustworthy Machine Learning 2026

CMS Team ID: 65 [Team LV]

Name 1: Ananya Bhardwaz

Student ID 1: 7076153

Email ID 1: anbh00002@stud.uni-saarland.de

Name 2: Aryan Aryan

Student ID 2: 7089648

Email ID 2: arar00002@stud.uni-saarland.de

1 Introduction

The task is to do watermark forgery attack. We are given 200 clean target images and eight folders of 25 watermarked images, where each folder belongs to a different hidden watermarking scheme. The attacker does not know the watermarking algorithm or the detector. Our final goal is to estimate the watermark signal from the available watermarked examples and apply it to unrelated clean target images while keeping the visual distortion low.

This is very different from watermark removal. In removal, the attacker tries to make a watermarked image pass as clean. In forgery, the attacker tries to make a clean image pass as watermarked. A successful forgery is dangerous because the detector may falsely attribute an image to a source that never produced it. Therefore, the assignment tests whether watermark presence alone is reliable evidence of provenance.

2 Methodology and Results

2.1 Baseline and motivation

The template baseline alpha-blends a clean target image with a watermarked source image. This showed bad results because instead of isolating only the watermark signal it transfers the visible content from the source image. The resulting images shows perceptual distortion and do not improve detection. Therefore, we only used the baseline only as a check and focused on estimating a reusable watermark residual.

2.2 Statistical averaging attack

Our first method assumes that, within one watermark group, every watermarked image can be written as clean image content plus a shared watermark irregularity:

$$S_i \approx C_i + W, \quad (1)$$

where S_i is a watermarked source image, C_i is its unknown clean content, and W is the common watermark signal. Averaging the 25 images in the group gives

$$\bar{S} = \frac{1}{25} \sum_{i=1}^{25} S_i \approx \bar{C} + W. \quad (2)$$

The difficulty is that $\bar{S}$ still contains residual image content. We first tried blur-based content suppression, but it left visible ghosting and removed useful high-frequency signal. We then used clean-reference subtraction the mean of available clean images was resized to the same resolution and subtracted from the mean watermarked image,

$$W_{est} = \bar{S} - \bar{C}_{ref}. \quad (3)$$

The resulting residual was scaled and added to the assigned clean target images. This method is simple and computationally cheap.

2.3 Split-half diagnostic

To check whether averaging was meaningful, we considered a split-half consistency diagnostic. The idea is to divide each watermark group into two subsets, estimate a watermark residual from each subset, and compare both estimates using correlation. High correlation would show a stable shared watermark pattern, while low correlation would suggest that the extracted residual is mostly image-specific noise. This means simple averaging is expected to work mainly for watermark groups with a reusable shared pattern, but it can fail or pass when the watermark is content-adaptive.

2.4 Preference-model gradient extraction

To attack the harder groups, we used a pretrained ConvNeXt-based image preference model as a different objective. The model was loaded on GPU and used to score whether an image looks natural. Instead of knowing the watermark detector, we optimized an additive perturbation using gradients from this model.

For a watermarked image x_w , we introduced a learnable perturbation δ and optimized

$$\hat{x} = \arg \max_{\delta} R(x_w + \delta), \quad (4)$$

where R is the preference model. The estimated watermark residual was then computed as

$$\hat{W} = x_w - \hat{x}. \quad (5)$$

The intuition is that optimization moves the image toward what the model considers more natural; the removed residual contains watermark-like artifacts. This is more flexible than plain averaging because it does not require the exact same perturbation to appear in every image.

2.5 Optimization details

All images were converted to RGB, resized to 768×768 , and optimized as PyTorch tensors. For each image, a perturbation was updated by gradient descent using the pretrained preference model as the objective. To reduce runtime, the perturbation from the previous image was used as a start for the next image. We also averaged the final optimization steps to reduce noise and updated the group estimate using an exponential moving average. Finally, the 25 extracted perturbations were combined into one group watermark.

2.6 Results

The experiments showed a clear difference between the tested methods. The direct alpha-blending baseline was not a reliable forgery method because it transferred visible source-image content and damaged perceptual quality. Averaging-based extraction was more meaningful because it attempted to isolate a shared residual watermark instead of copying the entire source image.

However, the split-half diagnostic showed that not all watermark groups contained a stable additive pattern. This explains why averaging was useful only for some groups and weak for others, especially the likely content-adaptive schemes. The preference-model extraction became the strongest final method because it used gradient information to recover watermark-like residuals even when simple statistical averaging was not reliable.

3 Conclusion

This assignment shows that watermark security depends strongly on the embedding strategy. Watermarks can be recovered by averaging multiple watermarked images and subtracting an estimate of natural image content, making them vulnerable even without access to the detector or watermarking algorithm.

Content-adaptive watermarks are more resistant to simple averaging, but they are not fully secure. Our preference-model attack shows that generic pretrained models can still provide useful gradient signals for extracting watermark-like residuals. Therefore, watermark systems should be tested against broader black-box and optimization-based attacks, not only simple baselines. In practice, watermark detection should be combined with stronger provenance mechanisms such as cryptographic signatures, metadata, and access logs.

Code Repository

Code is available at: https://github.com/Ananya-Bhardwaz/TML26_A4_65

References

- P. Yang, H. Ci, Y. Song, and M. Z. Shou. *Can Simple Averaging Defeat Modern Watermarks?* NeurIPS 2024.
- Z. Dong, C. Shuai, Z. Ba, P. Cheng, Z. Qin, Q. Wang, and K. Ren. *WMCopier: Forging Invisible Image Watermarks on Arbitrary Images*. NeurIPS 2025.
- T. Soucek, S. Rebuffi, P. Fernandez, N. Jovanovic, H. Elsahar, V. Lacatusu, T. Tran, and A. Mourachko. *Transferable Black-Box One-Shot Forging of Watermarks via Image Preference Models*. NeurIPS 2025.